

Scribed by:

1. Madhav Maheshwari (2022MT61975)
2. Naman Kale (2022MT11960)
3. Tatsam Ranjan Sharma (2022MT61969)
4. Taru Singhal (2022MT11922)
5. Krish Singh (2022MT61303)

1 Overview

In the last lecture, we introduced the concept of Pushdown Automata (PDA). In this lecture, we will establish the fundamental equivalence between Non-Deterministic Pushdown Automata (nPDA) and Context-Free Languages (CFLs). We will provide a detailed proof showing how to construct an nPDA for any given Context-Free Grammar (CFG).

2 Equivalence of nPDAs and Context-Free Languages

The central theorem of this topic states that the class of languages accepted by nPDAs is precisely the class of context-free languages. This is established by two theorems.

Theorem 1. *For any context-free language L , there exists an nPDA M such that $L(M) = L$.*

Theorem 2. *If $L = L(M)$ for some nPDA M , then L is a context-free language.*

This lecture will focus on proving the first theorem by providing a method to construct an nPDA from a grammar.

3 Proof: Constructing an nPDA from a CFG in CNF

To prove Theorem 1, we will provide a constructive algorithm. We simplify the process by assuming the given Context-Free Grammar G is in **Chomsky Normal Form (CNF)**. This is a valid assumption because any CFL (that doesn't generate λ) has an equivalent grammar in CNF.

A grammar in CNF has production rules only of the form $A \rightarrow BC$ or $A \rightarrow a$. Our nPDA will simulate a leftmost derivation using these simple rule formats.

3.1 Formal Construction

Let the CFG in Chomsky Normal Form be $G = (V, T, S, P)$. We construct the nPDA $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ as follows:

- **States (Q):** A start state, a main processing state, and a final state. $Q = \{q_{start}, q_{loop}, q_{final}\}$
- **Input Alphabet (Σ):** The set of terminals from the grammar. $\Sigma = T$
- **Stack Alphabet (Γ):** The set of variables plus a stack marker. $\Gamma = V \cup \{z_0\}$
- **Initial and Final States:** $q_0 = q_{start}$ and $F = \{q_{final}\}$.
- **Transition Function (δ):**

1. **Initialization:** Push the start symbol S onto the stack.

$$\delta(q_{start}, \lambda, z_0) = \{(q_{loop}, Sz_0)\}$$

2. **Expansion ($A \rightarrow BC$):** For each rule of this form, pop A and push C then B .

$$\delta(q_{loop}, \lambda, A) = \{(q_{loop}, BC)\} \quad \text{for every rule } A \rightarrow BC \in P$$

3. **Matching ($A \rightarrow a$):** For each rule of this form, consume input a and pop A .

$$\delta(q_{loop}, a, A) = \{(q_{loop}, \lambda)\} \quad \text{for every rule } A \rightarrow a \in P$$

4. **Acceptance:** When input is consumed and the stack is empty (marker z_0 is on top), accept.

$$\delta(q_{loop}, \lambda, z_0) = \{(q_{final}, z_0)\}$$

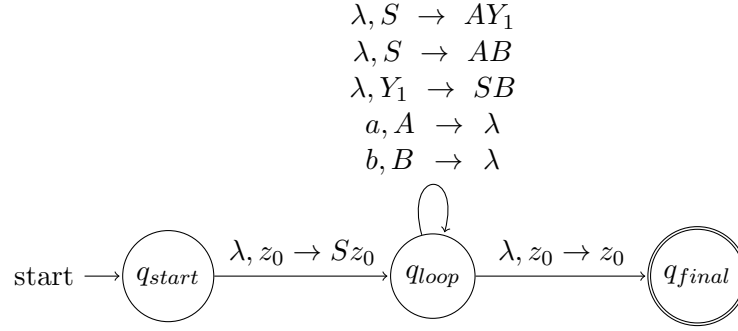
3.2 Example 1: The Language $L = \{a^n b^n \mid n \geq 1\}$

1. **Grammar in CNF:** The grammar $S \rightarrow aSb \mid ab$ is equivalent to the following CNF grammar:

- $S \rightarrow AY_1 \mid AB$
- $Y_1 \rightarrow SB$
- $A \rightarrow a$
- $B \rightarrow b$

2. **nPDA Construction:** The nPDA has $\Gamma = \{S, Y_1, A, B, z_0\}$ and the transitions are derived directly from the CNF rules.

3. Transition Graph Diagram:



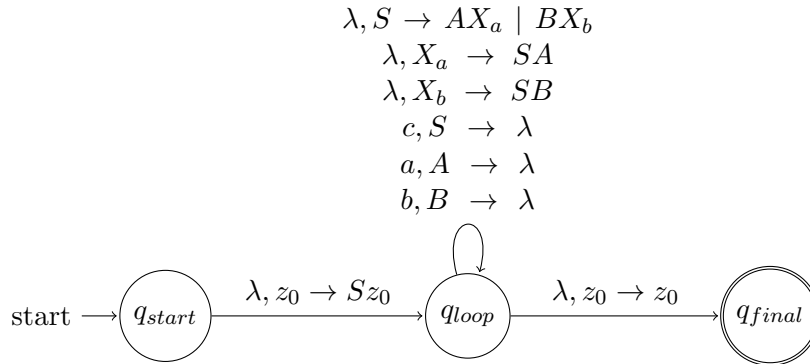
3.3 Example 2: The Language $L = \{wcw^R \mid w \in \{a, b\}^*\}$

1. Grammar in CNF: The grammar $S \rightarrow aSa \mid bSb \mid c$ is equivalent to the following CNF grammar:

- $S \rightarrow AX_a \mid BX_b \mid c$
- $X_a \rightarrow SA$
- $X_b \rightarrow SB$
- $A \rightarrow a$
- $B \rightarrow b$

2. nPDA Construction: The nPDA has $\Gamma = \{S, X_a, X_b, A, B, z_0\}$.

3. Transition Graph Diagram:



3.4 Handling the Empty String (λ)

The construction above assumes the language does not contain the empty string. It is trivial to expand the construction for cases where $\lambda \in L(G)$.

If the start symbol S of the grammar is nullable (meaning $S \Rightarrow^* \lambda$), we simply add a transition that allows the nPDA to move from its start state directly to a final state. The set of transitions from the start state becomes:

$$\delta(q_{start}, \lambda, z_0) = \{(q_{loop}, Sz_0), (q_{final}, z_0)\}$$

This allows the nPDA to non-deterministically choose to either begin parsing a non-empty string (by moving to q_{loop}) or to accept the empty string immediately (by moving to q_{final}).

4 A Language That is Not Context-Free

While context-free grammars are more powerful than regular grammars, they cannot describe all possible languages. A canonical example of a language that is not context-free is:

$$L = \{a^n b^n c^n \mid n \geq 0\}$$

This language requires matching the counts of three different symbols, which cannot be accomplished with a single stack. A pushdown automaton can use its stack to match the number of a 's with b 's, but it will have lost the count of a 's by the time it needs to match the c 's.

The proof that this language is not context-free is an important result, but it is beyond the scope of this lecture. It relies on a tool called the **Pumping Lemma for Context-Free Languages**.

References

- [1] Peter Linz, *An Introduction to Formal Languages and Automata, Fifth Edition*, Jones & Bartlett Learning, 2012.